# Urban Company Clone

Complete Project Documentation

| | | | | | |
|---|---|---|---|---|---|
| React 19 | Node.js | MongoDB | Firebase Auth | Socket.io | Razorpay |

Full-stack home services marketplace — React + Node.js + MongoDB

Version 2.0.0 • June 2026

# 1 Project Overview

Urban Company Clone is a full-stack home services marketplace modelled on India's Urban Company platform. It connects customers who need home services (cleaning, AC repair, beauty, massage, plumbing, electrical, painting) with verified professionals, tracks bookings in real time, and provides role-specific dashboards for customers, professionals, and administrators.

**Live at:** http://localhost:5174 | **Frontend:** React 19 + Vite | **Backend:** Node.js + Express | **Database:** MongoDB Atlas

## Architecture at a Glance

| Layer | Technology | Default Port | Deployment Target |
|---|---|---|---|
| Frontend (SPA) | React 19 + Vite + Tailwind CSS | 5173 / 5174 | Vercel |
| Backend (REST API) | Node.js + Express + Socket.io | 5000 / 5001 | Render.com |
| Database | MongoDB + Mongoose ODM | — (cloud) | MongoDB Atlas |
| Authentication | Firebase Auth (email + Google) | — (cloud) | Firebase Console |
| Payments | Razorpay (test mode) | — (cloud) | Razorpay Dashboard |
| File Storage | Cloudinary (pro documents) | — (cloud) | Cloudinary |
| Emails | Resend.com | — (cloud) | Resend |

## Key Features

- **Customer portal:** Browse 19+ services, detailed service pages, 4-step booking wizard, live booking tracking
- **Professional portal:** Registration wizard, dashboard with online/offline toggle, earnings tracker
- **Admin panel:** KPI dashboard, user/pro/booking/service management
- **Real-time updates:** Socket.io-based booking status pushed to customer browser
- **Payments:** Razorpay integration with HMAC verification
- **Authentication:** Firebase Auth (email/password + Google OAuth), role-based access

## 2 Repository Layout

```
urban-company-clone/ ■■■ src/ React frontend source ■ ■■■ main.jsx Entry point ■ ■■■ App.jsx Stub (routing in AppRouter) ■ ■■■ index.css Global styles + Tailwind component layer ■ ■■■ routes/ ■ ■ ■■■ AppRouter.jsx All route definitions ■ ■ ■■■ NotFound.jsx ■ ■■■ pages/ ■ ■ ■■■ user/ Customer pages (10 pages) ■ ■ ■■■ professional/ Pro portal (5 pages) ■ ■ ■■■ admin/ Admin panel (5 pages) ■ ■■■ components/ ■ ■ ■■■ common/ Shared layout & UI (10 components) ■ ■ ■■■ reviews/ ReviewList, ReviewForm ■ ■ ■■■ payment/ RazorpayCheckout ■ ■ ■■■ notifications/ NotificationBell ■ ■ ■■■ search/ SmartSearch ■ ■■■ context/ ■ ■ ■■■ AuthContext.jsx ■ ■■■ config/ ■ ■ ■■■ firebase.js Firebase SDK init ■ ■■■ hooks/ ■ ■■■ useBookingStatus.js Socket.io booking tracker ■ ■■■ useNotifications.js ■■■ backend/ ■ ■■■ server.js Express app + Socket.io server ■ ■■■ config/ ■ ■ ■■■ db.js MongoDB connection ■ ■ ■■■ firebase.js Firebase Admin SDK init ■ ■■■ middleware/ ■ ■ ■■■ auth.js Token verification ■ ■ ■■■ role.js Role guards ■ ■■■ models/ Mongoose schemas (5 models) ■ ■■■ routes/ REST endpoints (8 route files) ■ ■■■ controllers/ ■ ■■■ services/ ■ ■■■ utils/ ■■■ .env Frontend env vars (Firebase + API URL) ■■■ tailwind.config.js ■■■ vite.config.js ■■■ vercel.json ■■■ DEPLOYMENT.md
```

# 3 Tech Stack — Full Detail

## Frontend Dependencies

| Package | Version | Purpose |
|---|---|---|
| react | 19.2.6 | UI framework |
| react-dom | 19.2.6 | DOM renderer |
| react-router-dom | 7.15.1 | Client-side routing (all page navigation) |
| firebase | 12.13.0 | Auth — email/password + Google OAuth |
| socket.io-client | 4.8.3 | Real-time booking status updates |
| react-leaflet | 5.0.0 | Interactive map for address picker |
| leaflet | 1.9.4 | Underlying map library |
| react-icons | 5.6.0 | All icons (Feather Icons via FiX prefix) |
| tailwindcss | 3.4.19 | Utility-first CSS framework |
| vite | 8.0.12 | Dev server and production bundler |
| @vitejs/plugin-react | 6.0.1 | React transform for Vite |
| postcss / autoprefixer | latest | CSS processing pipeline |

## Backend Dependencies

| Package | Version | Purpose |
|---|---|---|
| express | 4.19.2 | HTTP server and routing |
| mongoose | 8.3.4 | MongoDB ODM — schemas, validation, queries |
| socket.io | 4.7.5 | WebSocket server for real-time events |
| firebase-admin | 13.10.0 | Verify Firebase ID tokens server-side |
| razorpay | 2.9.2 | Payment order creation and signature verification |
| cloudinary | 2.2.0 | Professional document upload and storage |
| multer | 1.4.5 | Multipart/form-data file upload middleware |
| resend | 3.2.0 | Transactional email delivery |
| bcryptjs | 2.4.3 | Password hashing (legacy, auth via Firebase) |
| cors | 2.8.5 | Cross-origin resource sharing middleware |
| dotenv | 16.4.5 | Environment variable loading |
| nodemon | 3.1.0 | Dev auto-restart on file changes |

# 4 Authentication System

Authentication is Firebase-based on the frontend; the backend verifies the same Firebase ID token using Firebase Admin SDK. There is no separate session system — every API request carries a short-lived JWT issued by Firebase.

## Frontend Auth Flow

- User signs in via **signInWithEmailAndPassword** or **signInWithPopup** (Google OAuth)
- Firebase fires **onAuthStateChanged** — AuthContext catches it
- Firebase **idToken** is fetched and stored in React context state
- AuthContext immediately calls **GET /api/users/me** with the token to load the full MongoDB user record (which carries the **role** field — user / professional / admin)
- Context exposes: user (Firebase object), dbUser (MongoDB object), token, loading, isAuthenticated, role, logout, refreshToken

## Backend Token Verification

- **authenticate** middleware reads Authorization: Bearer <token>
- Calls firebase-admin.auth().verifyIdToken(token)
- Looks up MongoDB User by firebaseUid — attaches as req.user
- Returns 401 on invalid/expired token, 403 on suspended account
- **optionalAuth** — same but swallows errors, used for public endpoints that optionally enrich results

## Route Protection

| Guard | File | Behaviour |
|---|---|---|
| ProtectedRoute | src/components/common/ProtectedRoute.jsx | Shows spinner while loading; redirects to /login if not authenticated; preserves intended destination in location.state.from |
| RoleRoute | src/components/common/ProtectedRoute.jsx | Same as ProtectedRoute but also checks user role against allowedRoles[]; redirects to / if role does not match |

## User Roles

| Role | Access |
|---|---|
| user (default) | All customer pages — booking, profile, wallet, addresses, wishlist, booking history |
| professional | All user pages PLUS /pro/dashboard, /pro/bookings, /pro/earnings, /pro/profile |
| admin | All professional pages PLUS /admin/dashboard, /admin/users, /admin/professionals, /admin/bookings, /admin/services |

**Making a user admin:** In MongoDB Atlas, run: db.users.updateOne({ email: 'you@example.com' }, { $set: { role: 'admin' } })

# 5 Frontend Pages — Complete Reference

## 5.1 Customer (User) Pages

### */ — Home (src/pages/user/Home.jsx)*

The main landing page, publicly accessible.

- **Hero section:** Bold heading 'Home services at your doorstep', 3x2 category icon grid (Women's Salon, Men's Salon, Cleaning, Painting, AC Repair, Electrician), 2x2 photo mosaic
- **Stats bar:** 4.8★ rating, 12M+ customers, 100% verified pros, 60 min guarantee
- **7 horizontal-scroll service sections:** Most Booked, Cleaning Essentials, Offers & Discounts (banner cards), Appliance Repair, New & Noteworthy, Salon for Men, Home Repair
- **Pro CTA banner:** Dark rounded banner inviting professionals to register at /pro/register
- All service data is hardcoded static arrays; images from picsum.photos with seed strings for consistent rendering across reloads

### */services — Services (src/pages/user/Services.jsx)*

- 19 hardcoded services in responsive grid: 2 cols (mobile) → 3 → 4 → 5 (desktop)
- **Sticky category pill bar:** All / Cleaning / Beauty & Spa / AC & Appliance / Massage & Wellness / Electrician / Plumbing / Painting
- **Sort dropdown:** Most Popular / Price Low→High / Price High→Low / Highest Rated
- Filters applied reactively with useMemo; URL params ?search= and ?category= supported
- Empty state shows 'No services found' with 'Clear filters' button

### */service/:id — Service Detail (src/pages/user/ServiceDetail.jsx)*

- Hero image with back-button overlay and gradient
- Service title, star rating, review count, duration badge, 'Verified professional' badge
- **What's included** checklist with green check icons; **Not included** items with red X
- **Sticky booking card** (right column on desktop): plan selector (e.g. 1BHK/2BHK/3BHK), quantity stepper, price breakdown (convenience fee free), Book Now button
- **Customer Reviews:** overall rating, star distribution bars, sample review cards
- **FAQ accordion:** 4 questions, expand/collapse with chevron
- Full data exists for IDs 1, 5, 10, 14; others fall back to ID 1

### */booking/:serviceId — Booking Wizard (src/pages/user/Booking.jsx) [Auth required]*

4-step wizard with sticky progress bar and fixed bottom CTA:

| Step | Content | Validation |
|---|---|---|
| 0 — Plan | Select pricing tier (radio buttons with price) | Always enabled |
| 1 — Date & Time | 7 upcoming date pills + 12 time slot buttons (7AM–6PM) | Slot must be selected |
| 2 — Address | Line 1, Line 2, City, Pincode fields | Line1 and Pincode required |
| 3 — Payment | Pay online (card/UPI) or pay at doorstep (cash) + order summary | Always enabled |

- Sidebar summary card updates live as user fills each step
- On Confirm: 1.2s simulated loading, then navigates to /booking/confirmation/BK{timestamp}

### */booking/confirmation/:id — Booking Confirmation [Auth required]*

- Animated success banner (dark bg, green checkmark, booking ID)
- Service summary card (date, time, address, package, amount paid/due)
- **Live status tracker:** 5-step vertical timeline driven by Socket.io (useBookingStatus hook)
- Demo status controls in development mode (manually advance status)
- Professional card appears when status reaches 'assigned' or later
- Quick actions: Download receipt, Share booking details

### */my-bookings — Booking History [Auth required]*

- Dark header, search bar (filters by service or professional name)
- Tab bar: All / Upcoming / Completed / Cancelled
- Per-card actions: Rate (opens ReviewForm modal), Rebook, Cancel, Details
- Star rating display for already-rated completed bookings

### */login and /signup — Auth Pages (bare layout, no Navbar/Footer)*

- **Login:** Split layout — dark left panel with trust stats, right panel with Google OAuth + email/password form; redirects to location.state.from after login
- **Signup:** Same split — adds Full Name, Phone fields; live password strength meter (4 bars: red/yellow/green); handles email-already-in-use error

### */profile — User Profile [Auth required]*

- Dark header with avatar (Firebase photo or initial letter), name, email
- Stats: 12 bookings, 8 reviews, Rs 2.4K savings
- Edit mode: toggle inline form for name, phone, city; camera icon for avatar upload
- Menu: Account (Addresses, Wishlist, Reviews, Booking History), Preferences (Notifications, Privacy)
- Sign out button calls Firebase signOut and redirects to /login

### */wallet — UC Wallet [Auth required]*

- Balance card (Rs 1,240), total earned vs spent breakdown
- Add Money and Refer & Earn quick action cards
- **Wallet tab:** Promo code input (validates URBAN50/FIRST100/CLEAN200), transaction history with credit/debit icons
- **Offers tab:** Dashed-border promo cards with one-click copy-to-clipboard, expiry dates

### */addresses — Saved Addresses [Auth required]*

- Lists Home/Work/Other typed addresses with icons; default address highlighted in green
- Full CRUD: Add modal, Edit modal, Delete confirmation modal; 'Set as default' action

### */wishlist — Wishlist [Auth required]*

- 2-column grid of saved services with image, category, rating, price
- Trash icon removes service; Book Now navigates to service detail

## 5.2 Professional Portal Pages

All at /pro/*, accessible to roles: professional or admin.

### */pro/register — Professional Registration (public)*

| Step | Fields |
|---|---|
| 1 — Personal Info | Full name, Email, Phone, Years of experience, Bio (optional) |
| 2 — Skills & City | Multi-select service tags (10 options), City dropdown (10 cities), Areas/localities covered |
| 3 — Availability | Day toggles (Mon–Sun), Start/End time pickers, estimated weekly hours display |
| 4 — Documents | ID proof (required), Address proof, Skill certificate — drag-to-upload with file size display |

- After submit: success screen with 3-step verification timeline (BG check 24h, skills call 48h, first job 72h)
- Left panel (desktop): branding, perks, 'average Rs 35K–80K/month' claim

### */pro/dashboard — Professional Dashboard*

- Online/Offline toggle: hides pending requests when offline
- 4 stat cards: Today's earnings, Jobs this month, Avg rating, Acceptance rate
- New requests list with Accept/Decline buttons (0.7s simulated API call)
- Quick nav grid to Bookings, Earnings, Reviews
- Recent completed jobs list

### */pro/earnings — Earnings*

- Net earnings card with gross and 15% platform fee breakdown
- Period selector: This week / This month / Last month / All time
- CSS-only bar chart — proportional height bars, no chart library needed
- Stats: Jobs done, Average per job, Growth %; Transaction payout history list
- Request payout button

## 5.3 Admin Panel Pages

All at /admin/*, accessible to role: admin only.

### */admin/dashboard — Admin Dashboard*

- **4 KPI cards:** Total revenue (Rs 48.2L, +18%), Active users (24,180), Professionals (1,842), Bookings today (312, -3%) with trend arrows
- **Weekly bar chart:** Toggle between Bookings count view and Revenue view (CSS-only)
- **Pending approvals panel:** New pro applications with approve/reject action buttons
- **Recent bookings table:** Booking ID, Service, Customer, Pro, Amount, Status badge
- **Quick nav cards:** Manage Users (24,180), Professionals (1,842), All Bookings (8,491), Services (47)

### *Other Admin Pages*

- **/admin/users:** User table with search, role badges, activation toggles
- **/admin/professionals:** Pro table with status (pending/approved/rejected/suspended), approve/reject actions
- **/admin/bookings:** Full platform booking list with status management and filters
- **/admin/services:** CRUD interface for the service catalogue

# 6 Components & Hooks

## 6.1 Common Components (src/components/common/)

| Component | Description |
|---|---|
| MainLayout.jsx | Shell layout — Navbar + Outlet + Footer, wraps all routes except /login, /signup, /pro/register. ToastProvider scoped inside. |
| Navbar.jsx | Sticky top bar: UC logo, nav links, location pill, search bar with autocomplete, cart icon, auth state (UserDropdown vs login link), mobile hamburger. UserDropdown shows profile links + sign out. |
| ProtectedRoute.jsx | ProtectedRoute: spinner while loading, redirects to /login. RoleRoute: also checks allowedRoles[]. |
| Toast.jsx | Full toast system. ToastProvider exposes .success/.error/.warning/.info methods. ToastItem: slide-in, progress bar, auto-dismiss 4s, portal in bottom-right (z-9999). |
| BookingTracker.jsx | Real-time 5-step status timeline component for BookingConfirmation page. |
| MapPicker.jsx | Leaflet map component for selecting service address coordinates. |
| Skeleton.jsx | Pulse skeleton loaders for card, text, and image placeholders. |
| Spinner.jsx | Centered circular loading spinner. |
| ErrorBoundary.jsx | Global React error boundary — catches render errors. |
| Footer.jsx | Site footer with category links, company info, social links. |

## 6.2 Other Components

| Component | Purpose |
|---|---|
| reviews/ReviewList.jsx | Displays overall rating, star distribution bars (5★ 76%, 4★ 19%...), individual review cards with helpful counts |
| reviews/ReviewForm.jsx | Star rating picker (1–5) + comment textarea for post-booking review submission |
| payment/RazorpayCheckout.jsx | Lazy-loads Razorpay SDK, creates order, opens payment popup, verifies HMAC signature, calls onSuccess/onFailure callbacks |
| notifications/NotificationBell.jsx | Bell icon with unread badge, dropdown of notification items |
| search/SmartSearch.jsx | Advanced search with live suggestions and category filtering |

## 6.3 Custom Hooks

| Hook | Returns | Description |
|---|---|---|
| useBookingStatus(bookingId, initialStatus) | { status, connected } | Opens Socket.io connection, joins room booking_{id}, listens for booking_status_changed events, returns reactive status string |
| useNotifications() | notifications state | Manages in-app notification list |

### *Status Labels (exported from useBookingStatus)*

| Status | Label | Colour |
|---|---|---|
| pending | Booking Received | Yellow |
| confirmed | Pro Assigned | Blue |
| on_the_way | Pro On The Way | Indigo |
| in_progress | Service In Progress | Orange |
| completed | Service Completed | Green |
| cancelled | Booking Cancelled | Red |

# 7 Design System

## 7.1 Colour Palette

| Token | Hex | Usage |
|---|---|---|
| brand.DEFAULT | #1AB64F | Primary CTA buttons, checkmarks, progress bars, active states |
| brand.light | #e8f9ef | Light backgrounds for selected items, info boxes |
| brand.dark | #148a3c | Hover state for brand-coloured elements |
| gray-900 | #111827 | Primary text, dark buttons, navbar background |
| gray-600 | #4B5563 | Secondary text, icons |
| gray-100 | #F3F4F6 | Card backgrounds, input backgrounds |

## 7.2 Typography

- **Font:** Inter (Google Fonts, weights 300–900) via @import in index.css
- Anti-aliased via -webkit-font-smoothing: antialiased
- Text selection colour overridden to brand green background

## 7.3 Tailwind Custom Classes (in index.css @layer components)

| Class | Purpose |
|---|---|
| .btn-primary | Green filled button — brand bg, white text, hover:brand-dark, active:scale-95 |
| .btn-outline | Gray border button — transparent bg, hover:gray-50 |
| .btn-dark | Black filled button — gray-900 bg, hover:gray-800 |
| .card | White rounded-2xl card with shadow-card border |
| .card-hover | Same as .card but lifts -translate-y-0.5 on hover |
| .input | Styled form input — rounded-xl, brand focus ring (ring-2 ring-brand/20) |
| .skeleton | animate-pulse bg-gray-100 — loading placeholder |
| .scroll-hide | Hides scrollbar on overflow-x containers (horizontal scroll sections) |
| .scroll-thin | 4px thin styled scrollbar for tall lists |
| .line-clamp-1/2/3 | Multi-line text truncation using -webkit-line-clamp |
| .modal-backdrop | Fixed overlay with backdrop-blur-sm for modals |

## 7.4 Shadows & Radii

| Token | Value | Usage |
|---|---|---|
| shadow-card | 0 1px 3px rgba(0,0,0,0.08), 0 1px 2px rgba(0,0,0,0.04) | Default cards |
| shadow-hover | 0 4px 16px rgba(0,0,0,0.12) | Elevated dropdowns, modals |
| shadow-nav | 0 1px 0 rgba(0,0,0,0.08) | Navbar bottom border |
| rounded-2xl | 16px | Cards, buttons, inputs |
| rounded-3xl | 24px | Large modals, hero cards |

## 7.5 Custom Animations

| Name | Duration | Usage |
|---|---|---|
| animate-toast-shrink | 4000ms (matches duration prop) | Toast progress bar countdown — shrinks from 100% to 0% width |
| animate-fade-in | 0.2s ease-out | Elements appearing (dropdowns, overlays) |
| animate-slide-up | 0.3s ease-out | Bottom sheets, suggestion dropdowns |
| animate-spin-slow | 2s linear infinite | Loader rings |

# 8 Backend Architecture

## 8.1 Server Entry Point (backend/server.js)

- Creates Express app and http.Server so Socket.io and Express share one port
- CORS configured to allow requests from FRONTEND_URL (default http://localhost:5173)
- JSON body parsing with 10 MB limit; URL-encoded body parsing
- Routes mounted at /api/auth, /api/users, /api/services, /api/bookings, /api/professionals, /api/reviews, /api/payments
- Global error handler returns { success: false, message }
- **Graceful start:** If MongoDB is unreachable, server still starts (routes that need DB return 503)

## 8.2 Socket.io Events

| Direction | Event | Payload | Effect |
|---|---|---|---|
| Client → Server | join_booking | bookingId (string) | Joins Socket.io room 'booking_{id}' |
| Client → Server | update_booking_status | { bookingId, status } | Broadcasts to room — used in demo |
| Server → Client | booking_status_changed | { bookingId, status } | Emitted when pro updates status via PATCH /api/bookings/:id/status |

## 8.3 REST API Endpoints

### *Auth — /api/auth*

| Method | Path | Auth | Description |
|---|---|---|---|
| POST | /register | None | Create user in MongoDB after Firebase signup. Idempotent — returns existing user if firebaseUid matches. |
| GET | /me | Bearer | Verify token, return current user object from MongoDB |

### *Users — /api/users*

| Method | Path | Auth | Description |
|---|---|---|---|
| GET | /me | Bearer | Get current user's full profile |
| PATCH | /me | Bearer | Update profile (name, phone, city, avatar) |
| GET | / | Admin | List all platform users |
| PATCH | /:id/role | Admin | Change a user's role |

### *Services — /api/services*

| Method | Path | Auth | Description |
|---|---|---|---|
| GET | / | None | List all active services; optional ?category= filter |
| GET | /:id | None | Get single service by ID |
| POST | / | Admin | Create new service |
| PATCH | /:id | Admin | Update existing service |
| DELETE | /:id | Admin | Deactivate service (soft delete) |

### *Bookings — /api/bookings*

| Method | Path | Auth | Description |
|---|---|---|---|
| POST | / | User | Create booking — user, service, scheduledAt, address, pricingTier, payment.method |
| GET | /my | User | User's own bookings, populated with service + professional |
| GET | /pro | Professional | Professional's assigned bookings, sorted by scheduledAt asc |
| PATCH | /:id/status | Professional | Update status — emits booking_status_changed via Socket.io |
| PATCH | /:id/cancel | User | Cancel a booking (must be user's own booking; cannot cancel completed/already-cancelled) |

### *Professionals — /api/professionals*

| Method | Path | Auth | Description |
|---|---|---|---|
| GET | / | None | List all approved professionals |
| POST | / | User | Submit professional application |
| PATCH | /:id/status | Admin | Approve / reject / suspend a professional |
| PATCH | /me/online | Pro | Toggle isOnline status |

### *Reviews — /api/reviews*

| Method | Path | Auth | Description |
|---|---|---|---|
| POST | / | User | Submit review (booking must be in completed status) |
| GET | /service/:serviceId | None | Get all reviews for a specific service |

### *Payments — /api/payments*

| Method | Path | Auth | Description |
|---|---|---|---|
| POST | /create-order | User | Create Razorpay order — returns orderId, amount in paise, currency, keyId |
| POST | /verify | User | Verify HMAC-SHA256 signature of orderID\|paymentID against secret key |

### *Health — /api/health*

| Method | Path | Auth | Description |
|---|---|---|---|
| GET | / | None | Returns { status: 'OK', timestamp: ISO-string } — used by Render health checks |

# 9 Database Models

## User Model (backend/models/User.js)

| Field | Type | Notes |
|---|---|---|
| firebaseUid | String (unique, indexed) | Links to Firebase Auth UID — primary lookup key for auth |
| name | String (required, trim) | Display name |
| email | String (unique, lowercase) | From Firebase — also stored in MongoDB for querying |
| phone | String | Optional mobile number |
| role | Enum: user \| professional \| admin | Default: user |
| avatar | String | URL (Firebase photo or custom upload) |
| city | String | User's primary city |
| addresses[] | { label, address, lat, lng, isDefault } | Saved delivery addresses |
| isActive | Boolean | False = account suspended; blocks authentication |
| timestamps | createdAt, updatedAt | Auto-managed by Mongoose |

## Professional Model (backend/models/Professional.js)

| Field | Type | Notes |
|---|---|---|
| user | ObjectId → User | One-to-one — the Professional is also a User |
| services[] | ObjectId[] → Service | Services this professional is qualified to perform |
| bio | String | Short description |
| experience | Number | Years of experience |
| city | String (required) | Primary operating city |
| areas[] | String[] | Serviceable neighbourhoods |
| rating | Number | Computed average rating |
| totalReviews | Number | Running count |
| totalEarnings | Number | Cumulative gross earnings (in rupees) |
| status | Enum: pending \| approved \| rejected \| suspended | Default: pending — admin must approve |
| documents | { idProof, addressProof, certificate } | Cloudinary URLs after upload |
| availability | { mon…sun booleans, startTime, endTime } | Working schedule |

| Field | Type | Notes |
|---|---|---|
| isOnline | Boolean | Real-time availability toggle |

## Service Model (backend/models/Service.js)

| Field | Type | Notes |
|---|---|---|
| title | String (required, trim) | Service display name |
| slug | String (unique) | URL-friendly identifier |
| category | String (required) | cleaning / beauty / ac-repair / wellness / electrical / plumbing / painting |
| description | String | Full description |
| icon | String | Emoji icon (default: wrench) |
| image | String | Cover image URL |
| duration | String | Human-readable e.g. '3-4 hrs' |
| includes[] | String[] | What's included bullet points |
| excludes[] | String[] | What's not included |
| pricing[] | { name, price, description } | Pricing tiers (1BHK/2BHK, 60min/90min, etc.) |
| rating | Number | Computed average across all reviews |
| totalReviews | Number | Review count |
| isActive | Boolean | Soft delete flag |
| tag | String | Bestseller / Trending / Popular (nullable) |

## Booking Model (backend/models/Booking.js)

| Field | Type | Notes |
|---|---|---|
| user | ObjectId → User | Customer who booked |
| professional | ObjectId → Professional | Null until assigned by system/admin |
| service | ObjectId → Service | Which service was booked |
| pricingTier | { name, price } | Snapshot of selected tier at booking time |
| status | Enum (6 values) | pending → confirmed → on_the_way → in_progress → completed \| cancelled |
| scheduledAt | Date (required) | Requested service datetime |
| address | { line1, city, lat, lng } | Service location |
| payment.method | Enum: online \| cash | Payment method chosen |

| Field | Type | Notes |
|---|---|---|
| payment.status | Enum: pending \| paid \| refunded | Payment state |
| payment.razorpayOrderId / PaymentId | String | From Razorpay API |
| payment.amount | Number | Amount in rupees |
| cancelReason | String | User-provided reason on cancellation |
| completedAt | Date | Set when status transitions to completed |

## Review Model (backend/models/Review.js)

| Field | Type | Notes |
|---|---|---|
| booking | ObjectId → Booking | Ensures review is tied to a real completed booking |
| user | ObjectId → User | Reviewer |
| professional | ObjectId → Professional | Who the review is about |
| service | ObjectId → Service | Which service the review covers |
| rating | Number (1–5, required) | Star rating |
| comment | String | Review text |
| isVerified | Boolean (default: true) | All reviews from this API are booking-verified |

# 10 Payment Flow — Razorpay

The payment flow uses a server-side order creation model to prevent amount tampering, and HMAC-SHA256 signature verification to confirm genuine Razorpay payments.

| Step | Actor | Action |
|---|---|---|
| 1 | User | Clicks 'Confirm & Pay Rs X' on booking wizard step 3 |
| 2 | Frontend | POST /api/payments/create-order { amount, bookingId } |
| 3 | Backend | Calls razorpay.orders.create({ amount in paise, currency: 'INR' }) |
| 4 | Backend | Returns { orderId, amount, currency, keyId } |
| 5 | Frontend | Lazily loads Razorpay checkout.js SDK via dynamic script tag |
| 6 | Frontend | Opens Razorpay popup with order details and theme colour #1AB64F |
| 7 | User | Completes payment inside Razorpay modal |
| 8 | Razorpay | Calls handler({ razorpay_order_id, razorpay_payment_id, razorpay_signature }) |
| 9 | Frontend | POST /api/payments/verify { signature, orderId, paymentId, bookingId } |
| 10 | Backend | Computes HMAC-SHA256 of orderID\|paymentID using RAZORPAY_KEY_SECRET |
| 11 | Backend | Compares computed vs received signature — returns 400 on mismatch |
| 12 | Frontend | Calls onSuccess callback; shows booking confirmation |

**Test card:** 4111 1111 1111 1111 | Any expiry and CVV | No real money in test mode

# 11 Real-Time Booking Tracking

Socket.io enables live booking status updates to be pushed to the customer browser without polling.

## Flow Diagram

```
Customer Browser Professional Browser | | useBookingStatus(bookingId) PATCH
/api/bookings/:id/status | | socket.connect(API_URL) Route handler updates DB status | |
socket.emit('join_booking', id) io.to('booking_{id}') | .emit('booking_status_changed')
|■■■■■■■■■■■■■■■ WebSocket event ■■■■■■■■■■■■■■■■■■| | setStatus(data.status) | UI re-renders
live status tracker
```

## Status Progression

| Status | Meaning | Visual |
|---|---|---|
| pending | Booking received, awaiting professional assignment | Yellow dot |
| confirmed | Professional has been assigned | Blue dot |
| on_the_way | Professional is travelling to the service address | Indigo dot |
| in_progress | Service has started | Orange dot |
| completed | Service successfully completed | Green dot |
| cancelled | Booking was cancelled | Red dot |

**Room isolation:** Each booking gets its own Socket.io room 'booking_{id}'. Status updates are only broadcast to the specific customer's browser, not all connected users.

# 12 Environment Variables

## Frontend (.env at project root)

| Variable | Example Value | Purpose |
|---|---|---|
| VITE_FIREBASE_API_KEY | AIzaSyD... | Firebase web API key |
| VITE_FIREBASE_AUTH_DOMAIN | project.firebaseapp.com | Firebase auth domain |
| VITE_FIREBASE_PROJECT_ID | urban-clone-7016 | Firebase project identifier |
| VITE_FIREBASE_STORAGE_BUCKET | project.firebasestorage.app | Firebase storage bucket |
| VITE_FIREBASE_MESSAGING_SENDER_ID | 185498787946 | Firebase messaging sender ID |
| VITE_FIREBASE_APP_ID | 1:185498...:web:b57... | Firebase app identifier |
| VITE_API_URL | http://localhost:5001/api | Backend API base URL |

## Backend (backend/.env)

| Variable | Example / Format | Purpose |
|---|---|---|
| PORT | 5000 | Express server port |
| NODE_ENV | development | Environment mode |
| MONGODB_URI | mongodb+srv://user:pass@cluster.../dbname | MongoDB Atlas connection string |
| FIREBASE_PROJECT_ID | urban-clone-7016 | Firebase Admin SDK — project |
| FIREBASE_CLIENT_EMAIL | firebase-adminsdk@project.iam.gserviceaccount.com | Firebase Admin SDK — service account |
| FIREBASE_PRIVATE_KEY | "-----BEGIN PRIVATE KEY-----\n..." | Firebase Admin SDK — private key (JSON escaped) |
| CLOUDINARY_CLOUD_NAME | yourcloud | Cloudinary cloud identifier |
| CLOUDINARY_API_KEY | 123456789 | Cloudinary API key |
| CLOUDINARY_API_SECRET | abc123secret | Cloudinary API secret |
| RAZORPAY_KEY_ID | rzp_test_XXXX | Razorpay API key (test mode) |
| RAZORPAY_KEY_SECRET | XXXXXXXXXXX | Razorpay secret for HMAC verification |
| RESEND_API_KEY | re_XXXXXXXXXX | Resend transactional email API key |
| RESEND_FROM | noreply@yourdomain.com | Sender email address |
| FRONTEND_URL | http://localhost:5173 | Allowed CORS origin |

| Variable | Example / Format | Purpose |
| --- | --- | --- |
| JWT_SECRET | super_secret... | Legacy field (not actively used) |

# 13 Complete Routing Map

| Path | Component | Access | Layout |
|---|---|---|---|
| / | Home | Public | MainLayout |
| /services | Services | Public | MainLayout |
| /services/:category | Services | Public | MainLayout |
| /service/:id | ServiceDetail | Public | MainLayout |
| /login | Login | Public | Bare |
| /signup | Signup | Public | Bare |
| /booking/:serviceId | Booking | Auth | MainLayout |
| /booking/confirmation/:id | BookingConfirmation | Auth | MainLayout |
| /profile | Profile | Auth | MainLayout |
| /my-bookings | BookingHistory | Auth | MainLayout |
| /wallet | Wallet | Auth | MainLayout |
| /addresses | Addresses | Auth | MainLayout |
| /wishlist | Wishlist | Auth | MainLayout |
| /pro/register | ProRegister | Public | Bare |
| /pro/dashboard | ProDashboard | Pro / Admin | No Layout |
| /pro/bookings | ProBookings | Pro / Admin | No Layout |
| /pro/earnings | ProEarnings | Pro / Admin | No Layout |
| /pro/profile | ProProfile | Pro / Admin | No Layout |
| /admin/dashboard | AdminDashboard | Admin only | No Layout |
| /admin/users | AdminUsers | Admin only | No Layout |
| /admin/professionals | AdminProfessionals | Admin only | No Layout |
| /admin/bookings | AdminBookings | Admin only | No Layout |
| /admin/services | AdminServices | Admin only | No Layout |
| * | NotFound | Public | — |

# 14 Deployment Architecture

## Infrastructure Overview

| Service | Provider | Purpose | Config File |
|---|---|---|---|
| Frontend | Vercel | Serves React SPA as static files; all paths rewrite to index.html for client routing | vercel.json |
| Backend API | Render.com | Runs Node.js server; auto-deploys from GitHub; free tier spins down after 15min idle | backend/render.yaml |
| Database | MongoDB Atlas | M0 free cluster; network access set to 0.0.0.0/0 for Render dynamic IPs | — |
| Auth | Firebase | Token issuance only — no Firestore/Storage used | — |
| Files | Cloudinary | Professional document uploads (ID proof, certificates) | — |
| Email | Resend.com | Booking confirmations, pro approval notifications (3000 emails/month free) | — |

## Third-Party Service Setup Checklist

| Service | Steps | Keys Needed |
|---|---|---|
| MongoDB Atlas | Create free M0 cluster → DB user → Allow 0.0.0.0/0 → Copy connection string | MONGODB_URI |
| Firebase Auth | Create project → Enable Email/Password + Google → Generate service account JSON → Add Web app | 6 VITE_FIREBASE_* vars + 3 FIREBASE_* backend vars |
| Cloudinary | Sign up → Dashboard → Copy Cloud name, API Key, Secret | CLOUDINARY_CLOUD_NAME, API_KEY, API_SECRET |
| Razorpay | Sign up → Settings → API Keys → Generate Test Keys | RAZORPAY_KEY_ID, RAZORPAY_KEY_SECRET |
| Resend | Sign up → API Keys → Create → Add/verify domain | RESEND_API_KEY, RESEND_FROM |

## Post-Deploy Checklist

- MongoDB Atlas cluster live, connection string working
- Firebase Auth enabled (Email + Google), service account key configured in backend
- Render backend health check: GET /api/health returns 200
- Vercel frontend loads, can navigate all pages
- Test user registration + login (email and Google)
- Test full booking creation flow end-to-end
- Razorpay test payment: card 4111 1111 1111 1111
- Email confirmation arrives after booking

- Admin account: set role: 'admin' directly in MongoDB Atlas for test user
- Pro account: register via /pro/register, approve in Admin panel

# 15 Local Development

## Quick Start

```
# 1. Install frontend dependencies cd urban-company-clone npm install # 2. Start frontend (Vite dev server) npm run dev # → http://localhost:5173 # 3. In a second terminal, start backend cd urban-company-clone/backend npm install npm run dev # uses nodemon # → http://localhost:5000
```

**Note:** The frontend works fully standalone without the backend running. All service catalogue data, booking flows, and admin panels use hardcoded static data. Only auth-dependent features (profile save, real bookings) require the backend.

## Available Scripts

| Location | Script | Command | Result |
|---|---|---|---|
| Frontend | dev | vite | HMR dev server on :5173 |
| Frontend | build | vite build | Production bundle to /dist |
| Frontend | preview | vite preview | Preview production build |
| Frontend | lint | eslint . | ESLint check |
| Backend | dev | nodemon server.js | Auto-restart dev server |
| Backend | start | node server.js | Production start (Render uses this) |

# 16 Current State vs. Full Integration

The frontend is a fully navigable, polished demo using hardcoded data. Backend integration points are built and ready but not yet wired up in the UI components.

| Feature | Frontend | Backend API | Notes |
|---|---|---|---|
| Firebase Auth (login/signup/Google) | Live | Live (verifies tokens) | Fully working end-to-end |
| Service catalogue (browse) | Static hardcoded | Full CRUD API ready | Replace static arrays with GET /api/services |
| Booking creation | Simulated (fake ID) | POST /api/bookings ready | Wire up booking wizard to real API |
| Real-time status tracking | Socket.io wired | Emits on status update | Works end-to-end once booking IDs are real |
| Booking history | Static sample data | GET /api/bookings/my ready | Replace static BOOKINGS array with API call |
| Razorpay payments | Full component built | Create-order + verify ready | Needs valid Razorpay keys in .env |
| Professional registration | Simulated submit | Full application API ready | Wire form submission to POST /api/professionals |
| Admin panel data | Static demo data | All routes ready | Replace static stats with GET /api/* calls |
| Profile save | Simulated 800ms | PATCH /api/users/me ready | Wire edit form to API |
| Review submission | Modal renders form | POST /api/reviews ready | Wire ReviewForm submit to API |
| Address management | Local state only | addresses[] on User model | Sync to PATCH /api/users/me addresses |
| Cloudinary uploads | File input only | Multer + Cloudinary ready | Wire document upload in pro registration |

## What's Needed to Go Production-Ready

- Set all environment variables in .env (both frontend and backend)
- Replace static data arrays in pages with fetch() calls to corresponding /api/* endpoints
- Wire the 4-step booking wizard to POST /api/bookings instead of navigating with a fake ID
- Connect RazorpayCheckout component to the booking confirmation step
- Add real data seeding for services in MongoDB (or build admin CRUD UI fully)
- Enable Cloudinary upload in professional registration step 4
- Add email notifications via Resend on booking creation and professional approval
- Set role: 'admin' in MongoDB Atlas for your first admin user

## Built with

| | | | |
|---|---|---|---|
| React 19 | Vite 8 | Tailwind CSS 3 | Node.js 18+ |
| MongoDB | Firebase Auth | Socket.io 4 | Razorpay |
| Cloudinary | Resend | React Router 7 | Leaflet |

Urban Company Clone — Project Documentation

Version 2.0.0 • June 2026 • Full-Stack Home Services Platform